Week 10 — Fairness and Robustness Audit

Course: DS5619 Machine Learning Systems Operations
Student ID: `142301024`
Seed: `2290012310` (salt: `week10`, selected sub-seed: `2290012319`)

What Was Implemented

In this lab, I built a fairness audit and adversarial robustness checking pipeline for the BMD-45 vehicle detector across 14 vehicle categories in `src/fairness_audit.py` and `src/adversarial.py`.

1. Fairness Audit (`src/fairness_audit.py`):

- **`compute_iou(box_a, box_b)`**: Computes the Intersection-over-Union (IoU) between two bounding boxes $[x, y, w, h]$. Clamps overlap dimensions to ≥ 0 so non-overlapping boxes cleanly return 0.0.
- **`match_ground_truth(gt_boxes, detections, iou_threshold=0.3)`**: Compares each ground-truth box against all detector predictions. Marks a ground-truth object as detected if any prediction has an IoU ≥ 0.3.
- **`audit_by_category(coco_path, fixtures_dir, ...)`**: Loads all 60 CCTV images, runs detections, matches annotations, and computes the True Positive Rate / Recall ($\text{detected} \ / \ \text{total}$) for every vehicle category.
- **`find_underperforming(category_stats, recall_threshold=0.9)`**: Flags any category falling below the 90% recall threshold, sorted worst-first.

2. Adversarial Robustness Check (`src/adversarial.py`):

- **`perturb_image(img, eps)`**: Simulates an FGSM-style targeted shift on pixel channels within an L_∞ budget ϵ (subtracts ϵ from the red channel and adds ϵ to green and blue, clamped to $[0, 255]$).
- **`audit_under_attack(coco_path, fixtures_dir, eps_values=...)`**: Runs the fairness audit across perturbation budgets $\epsilon \in \{0, 10, 20, 30, 40\}$.
- **`find_most_fragile(attack_stats, eps, top_k=3)`**: Calculates the absolute recall drop ($\text{recall}_{\text{clean}} - \text{recall}_{\epsilon}$) for each category and returns the top 3 most fragile classes.

Results

Running the pipeline (`python src/run_pipeline.py`) generated `fairness_report.json`:

1. Clean Fairness Audit (90% Threshold):

Out of 14 vehicle classes, 3 underperformed and were flagged: - **Two-wheeler:** 11/13 detected = **84.6%** recall *(FLAGGED)* - **Other:** 15/17 detected = **88.2%** recall *(FLAGGED)* - **Bicycle:** 16/18 detected = **88.9%** recall *(FLAGGED)*

All larger vehicle types passed comfortably: **Bus** (92.3%), **Truck** (92.6%), **Tempo-traveller** (91.7%), and **Hatchback / Sedan / SUV / MUV / Mini-bus** (100.0%).

Why small vehicles struggled: Bicycles and two-wheelers have small bounding box multipliers in the generator (0.35 and 0.45). With fewer total pixels, they often fail the detector's minimum connected-component pixel threshold (20 pixels), creating a structural disparity against micro-mobility users.

2. Adversarial Attack Findings:

As perturbation budget ϵ increased, detection recall degraded across all categories. At $\epsilon = 20$: - **#1 Most Fragile: Bicycle** (recall dropped from 88.9% to 50.0% = **-38.89% drop**) - **#2 Most Fragile: Other** (recall dropped from 88.2% to 58.8% = **-29.42% drop**) - **#3 Most Fragile: Bus** (recall dropped from 92.3% to 65.4% = **-26.93% drop**)

Key Takeaway:

Smaller vehicles already underperform in clean daylight, and they are the first to collapse when image quality degrades or adversarial color shifts occur. Auditing by subpopulation surfaces these disparities before deployment.